FG-Net Finetuning ¶

In [25]:

```
from tensorflow import keras

new_model = keras.Sequential(k_means_wiki.layers[:-2])
```

In [26]:

```
new_model.summary()
```

```
Model: "sequential_1"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 vgg16 (Functional)          (None, 7, 7, 512)         14714688

 flatten_1 (Flatten)         (None, 25088)             0

 dense_3 (Dense)             (None, 4000)              100356000

 dropout_1 (Dropout)         (None, 4000)              0

=================================================================
Total params: 115,070,688
Trainable params: 0
Non-trainable params: 115,070,688
_________________________________________________________________
```

In [32]:

```
# new_model.trainable = False ## Not trainable weights
dense_layer_1 = layers.Dense(2000, activation='relu')
dropout = layers.Dropout(0.5)
dense_layer_2 = layers.Dense(200, activation='relu')
prediction_layer = layers.Dense(18, activation='softmax')


fgnet_model = models.Sequential([
    new_model,
    dense_layer_1,
    dropout,
    dense_layer_2,
    dropout,
    prediction_layer
])
fgnet_model.summary()
```

```
Model: "sequential_3"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 sequential_1 (Sequential)   (None, 4000)              115070688

 dense_2 (Dense)             (None, 2000)              8002000

 dropout_1 (Dropout)         multiple                  0

 dense_3 (Dense)             (None, 200)               400200

 dense_4 (Dense)             (None, 18)                3618

=================================================================
Total params: 123,476,506
Trainable params: 8,405,818
Non-trainable params: 115,070,688
_________________________________________________________________
```

In [46]:

```
# fgnet data generator for vgg16 model pretrained on

from keras_preprocessing.image import ImageDataGenerator
import pandas as pd

# datagen=ImageDataGenerator(rescale=1./255.,validation_split=0.25)

train_df = pd.read_csv('/content/fgnet_final_file.csv') # for fgnet "/content/fgnet_wi
train_df['final_label'] = train_df['final_label'].astype(str)
# train_df['age_group'] = train_df['age_group'].astype(str)
# train_df['gender'] = train_df['gender'].astype(str)

# y_coll = ['age_group', 'gender']

train_datagen = ImageDataGenerator(
    rescale=1 / 255.0,
    validation_split=0.25
)

train_generator = train_datagen.flow_from_dataframe(
    dataframe=train_df,
    directory="/content/cropped_fgnet/",
    x_col="full_path",
    y_col="final_label",
    target_size=(224, 224),
    batch_size=16,
    class_mode="categorical",
    subset='training',
    shuffle=True,
    seed=42
)

valid_generator = train_datagen.flow_from_dataframe(
    dataframe=train_df,
    directory="/content/cropped_fgnet/",
    x_col="full_path",
    y_col="final_label",
    target_size=(224, 224),
    batch_size=16,
    class_mode="categorical",
    subset='validation',
    shuffle=True,
    seed=42
)

```

```
Found 711 validated image filenames belonging to 18 classes.
Found 236 validated image filenames belonging to 18 classes.
```

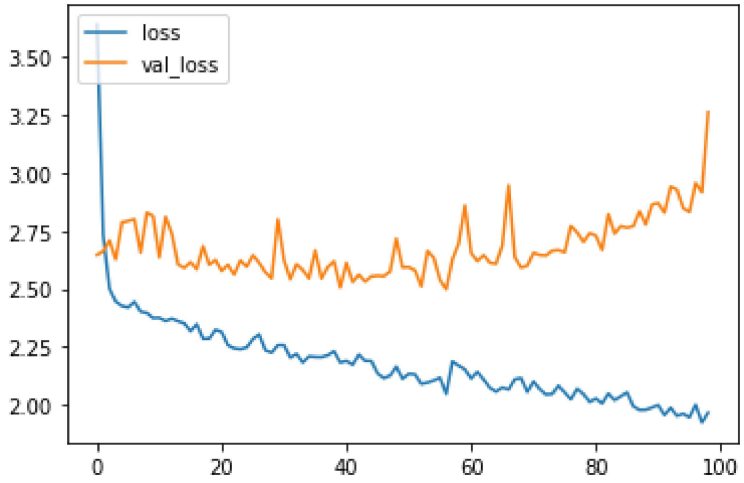
In [47]:

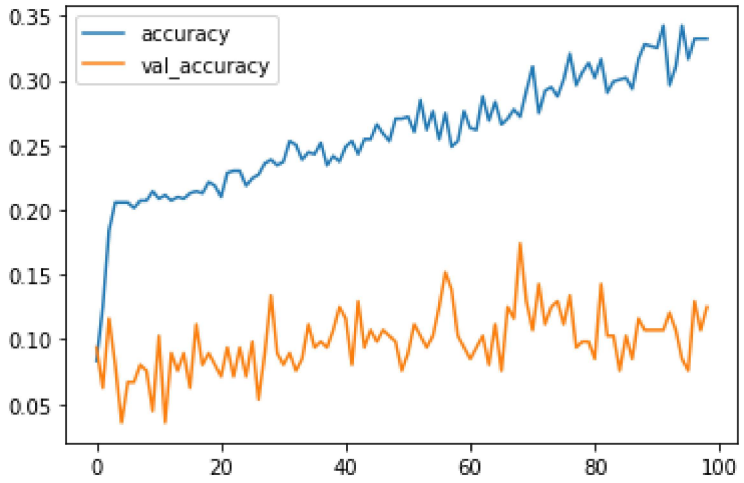
```
from tensorflow.keras.callbacks import EarlyStopping

fgnet_model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy'],
)


es = EarlyStopping(monitor='val_accuracy', mode='max', patience=30,  restore_best_weigh

history  = k_means_wiki.fit(train_generator,
                    validation_data = valid_generator,
                    steps_per_epoch = train_generator.n//train_generator.batch_size,
                    validation_steps = valid_generator.n//valid_generator.batch_size,
                    epochs=100, callbacks=[es])
```

```
Epoch 1/100
44/44 [==============================] - 9s 152ms/step - loss: 3.6385 - ac
curacy: 0.0835 - val_loss: 2.6470 - val_accuracy: 0.0938
Epoch 2/100
44/44 [==============================] - 6s 135ms/step - loss: 2.7305 - ac
curacy: 0.1252 - val_loss: 2.6630 - val_accuracy: 0.0625
Epoch 3/100
44/44 [==============================] - 6s 144ms/step - loss: 2.5006 - ac
curacy: 0.1842 - val_loss: 2.7098 - val_accuracy: 0.1161
Epoch 4/100
44/44 [==============================] - 6s 138ms/step - loss: 2.4461 - ac
curacy: 0.2058 - val_loss: 2.6276 - val_accuracy: 0.0804
Epoch 5/100
44/44 [==============================] - 6s 133ms/step - loss: 2.4270 - ac
curacy: 0.2058 - val_loss: 2.7866 - val_accuracy: 0.0357
Epoch 6/100
44/44 [==============================] - 6s 131ms/step - loss: 2.4206 - ac
curacy: 0.2058 - val_loss: 2.7927 - val_accuracy: 0.0670
Epoch 7/100
```

In [48]:

```
import matplotlib.pyplot as plt
# training loss and validation loss
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.legend(['loss', 'val_loss'], loc='upper left')
plt.show()
```



In [49]:

```
# training accuracy and validation accuracy
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.legend(['accuracy', 'val_accuracy'], loc='upper left')
plt.show()
```